

Subject : Compiler Constructor

Topic Title : Lexical Analyzer

1. Introduction

A **Lexical Analyzer**, also known as a **Scanner**, is the first phase of a compiler. Its primary function is to read the source program character by character and convert the input stream into a sequence of tokens. It identifies meaningful patterns in the source code and removes unnecessary elements such as spaces, tabs, new lines, and comments.

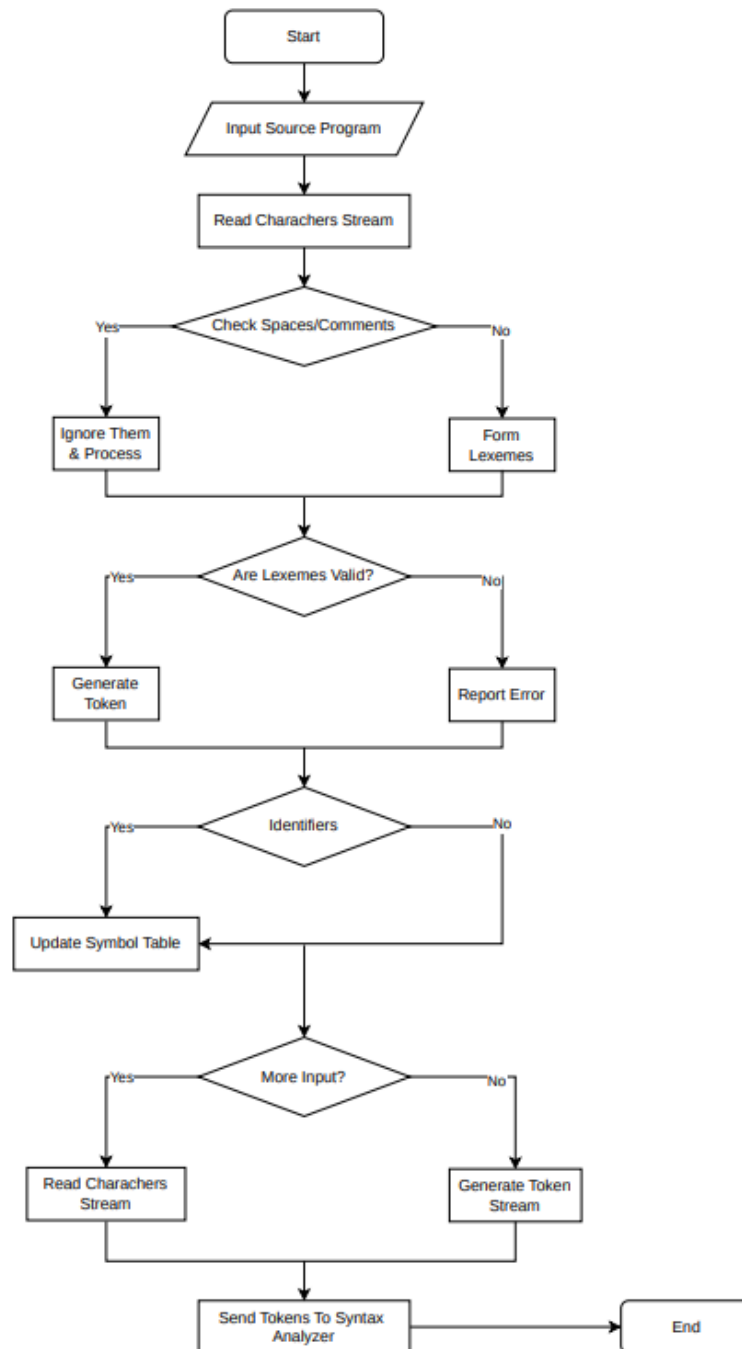
The lexical analyzer acts as an interface between the source program and the syntax analyzer.

2. Objectives

The main objectives of a lexical analyzer are:

- Read the source program.
- Scan the character stream.
- Identify lexemes.
- Generate tokens.
- Detect lexical errors.
- Maintain the symbol table.
- Remove spaces and comments.
- Pass token stream to the syntax analyzer.

3. Flowchart



4. Terminologies

1 Character Stream

A sequence of characters obtained from the source program that serves as input to the lexical analyzer.

2 Lexeme

A lexeme is the smallest meaningful sequence of characters identified during lexical analysis.

3 Token

A token is a categorized representation of a lexeme generated by the lexical analyzer.

4 Symbol Table

A data structure used to store information about identifiers encountered during lexical analysis.

5 Lexical Error

An error that occurs when a sequence of characters does not conform to the rules of the programming language.

5. Working of Lexical Analyzer

Step 1: Start

The lexical analyzer begins execution after receiving the source program.

Step 2: Input Source Program

The source code is provided as input to the lexical analyzer for processing.

Step 3: Read Character Stream

Characters from the source program are read sequentially.

Step 4: Check Spaces and Comments

The lexical analyzer checks whether the current input contains spaces, tabs, new lines, or comments.

If Yes

The spaces and comments are ignored because they do not contribute to token generation.

If No

The analyzer proceeds to form lexemes.

Step 5: Form Lexemes

Characters are grouped together into meaningful units known as lexemes.

Step 6: Validate Lexemes

The analyzer checks whether the formed lexeme follows the lexical rules of the programming language.

If Valid

Processing continues and a token is generated.

If Invalid

A lexical error is reported.

Step 7: Generate Token

A token corresponding to the valid lexeme is generated.

Tokens provide a simplified representation of the source program for subsequent compiler phases.

Step 8: Check Identifier

The analyzer determines whether the generated token represents an identifier.

If Yes

The identifier information is stored in the symbol table.

If No

Processing continues without updating the symbol table.

Step 9: Update Symbol Table

Information related to identifiers is recorded in the symbol table for future compiler phases.

The symbol table helps manage variable names, function names, and other user-defined identifiers.

Step 10: Check More Input

The analyzer determines whether more characters remain in the source program.

If Yes

The process returns to reading the next character stream.

If No

The complete token stream is generated.

Step 11: Generate Token Stream

All generated tokens are collected into a token stream.

The token stream represents the lexical structure of the source program.

Step 12: Send Tokens to Syntax Analyzer

The generated token stream is passed to the syntax analyzer, which performs syntactic analysis according to the grammar rules of the language.

Step 13: End

The lexical analysis process terminates after all characters have been processed and the token stream has been successfully forwarded to the syntax analyzer.

6. Functions of Lexical Analyzer

- Reads source code.
- Removes spaces and comments.
- Forms lexemes.
- Generates tokens.
- Detects lexical errors.
- Maintains symbol table.
- Improves compilation efficiency.
- Supplies tokens to the syntax analyzer.

7. Advantages

- Simplifies the compiler design.
- Increases compilation speed.
- Detects errors at an early stage.
- Reduces complexity for the syntax analyzer.
- Organizes identifier information efficiently.
- Provides a structured token stream.
- Improves overall compiler performance.

8. Applications

- Compiler Construction
- Interpreter Design
- Programming Language Processing
- Code Editors
- Integrated Development Environments (IDEs)
- Static Code Analysis Tools
- Source Code Validation Systems

9. Conclusion

The Lexical Analyzer is the first phase of the compilation process and plays a crucial role in converting source code into a structured token stream. It reads the character stream, eliminates unnecessary elements, identifies lexemes, generates tokens, maintains the symbol

table, reports lexical errors, and forwards the processed information to the syntax analyzer. By performing these tasks efficiently, the lexical analyzer provides a solid foundation for the subsequent phases of compilation.